



ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΕΠΕΞΕΡΓΑΣΤΩΝ & ΥΛΙΚΟΥ
ΕΡΓΑΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ ΓΙΑ ΤΟ ΜΑΘΗΜΑ:
Οργάνωση Υπολογιστών
ΗΡΥ 302
<http://www.mhl.tuc.gr>
ΕΑΡΙΝΟ ΕΞΑΜΗΝΟ 2024

Αναφορά Εργαστηριακής Άσκησης 2

Αριθμός Ομάδας: 36

Ονοματεπώνυμο 1: Μαλαμάς Νικόλαος (2020030180)

Ονοματεπώνυμο 2: Οικονόμου Χριστόφορος (2020030117)

Σκοπός της άσκησης

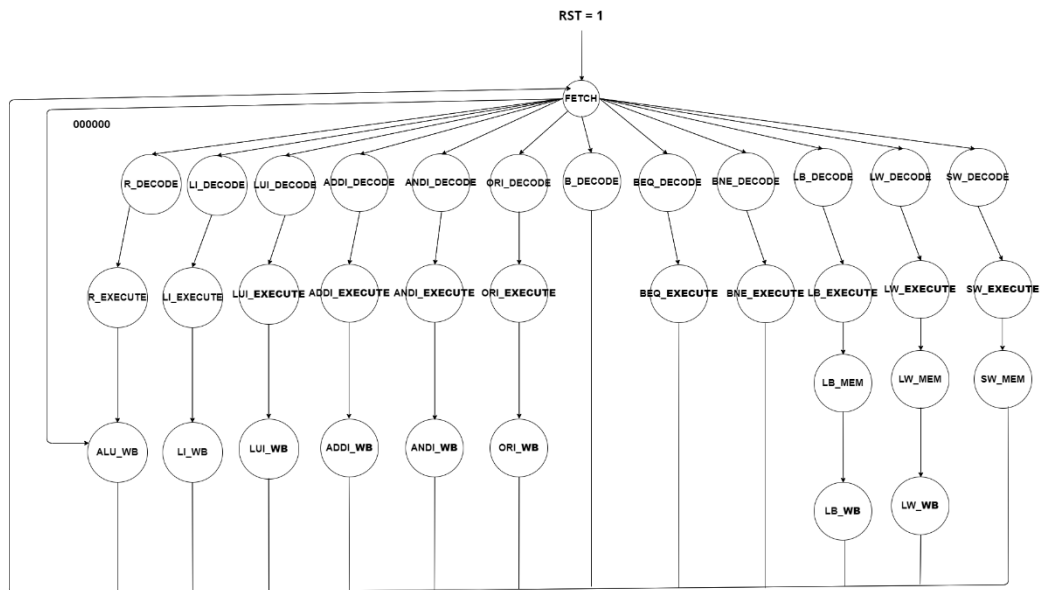
Η άσκηση αποσκοπούσε στην μετατροπή του επεξεργαστή ενός κύκλου που υλοποιήθηκε κατά την πρώτη φάση του Project, σε επεξεργαστή πολλαπλών κύκλων, ανάλογα με τις απαιτήσεις της εκάστοτε εντολής, που αυτός υποστηρίζει.

Περιγραφή της Σχεδίασης

Αρχικά, ανάμεσα από κάθε βαθμίδα του Datapath, τοποθετήθηκαν καταχωρητές ανάλογα με το πλήθος των εξόδων της βαθμίδας. Σκοπός της τοποθέτησης αυτών των καταχωρητών είναι κάθε έξοδος της βαθμίδας να αποθηκεύεται σε έναν από αυτούς και να εμφανίζεται στην έξοδό τους έναν κύκλο ρολογιού μετά. Κατά αυτόν τον τρόπο, μπορούμε να επιτύχουμε τον επιθυμητό διαχωρισμό εκτέλεσης της κάθε εντολής σε παραπάνω από έναν κύκλους.

Εν συνεχεία, η μονάδα ελέγχου του επεξεργαστή υπέστη πολλές αλλαγές συγκριτικά με το πως αυτή υλοποιήθηκε στον επεξεργαστή ενός κύκλου, έτσι ώστε να επιτύχει τον κατάλληλο συγχρονισμό μεταξύ των βαθμίδων και των εισόδων που αυτές λαμβάνουν. Πιο συγκεκριμένα, η μονάδα ελέγχου υλοποιήθηκε ως μία FSM που δέχεται σαν είσοδο της τον Opcode της κάθε εντολής, καθώς το βοηθητικό σήμα Zero, και στην έξοδο παράγει τα κατάλληλα σήματα ελέγχου για την εκτέλεση της εντολής, σε πολλαπλούς κύκλους. Είναι εμφανές ότι για τον κατάλληλο συγχρονισμό της με το Datapath του επεξεργαστή, η μονάδα ελέγχου δέχεται σαν είσοδο το ρολόι και το Reset σήμα.

Για αποδοτικότερο σχεδιασμό, ήταν απαραίτητη η ομαδοποίηση των R-type εντολών, καθώς αυτές απαιτούν σχεδόν πανομοιότυπα σήματα ελέγχου για να ολοκληρωθούν. Οι I-type εντολές λόγω του τρόπου με τον οποίο υλοοιήθηκαν, κατά την πρώτη φάση του Project, δεν μπόρεσαν να ομαδοποιηθούν, με αποτέλεσμα κάθε I-type εντολή να καταλαμβάνει ένα ξεχωριστό state της FSM, για κάθε βαθμίδα του Datapath που αυτή αξιοποιεί.



State Diagram του Control Unit

Για λόγους ευκρίνειας, οι μεταβάσεις και οι έξοδοι κάθε κατάστασης συνοψίζονται στον παρακάτω πίνακα:

FETCH	100000	R_DECODE	R_EXECUTE	ALU_WB	FETCH
0000000000000000	0000000000000000	000000000 & func(3 down to 0) & 00			0000000000000000
FETCH	111000	LI_DECODE	LI_EXECUTE	LI_WB	FETCH
0000000000000000	0000000000000001	0000000100000000			0000000000000000
FETCH	111001	LUI_DECODE	LUI_EXECUTE	LUI_WB	FETCH
0000000000000000	0000000000000011	0000000100000000			0000000000000000
FETCH	110000	ADDI_DECODE	ADDI_EXECUTE	ADDI_WB	FETCH
0000000000000000	0000000000000001	0000000100000000			0000000000000000
FETCH	110010	ANDI_DECODE	ANDI_EXECUTE	ANDI_WB	FETCH
0000000000000000	0000000000000000	0000000100010000			0000000000000000
FETCH	110011	ORI_DECODE	ORI_EXECUTE	ORI_WB	FETCH
0000000000000000	0000000000000000	0000000100011000			0000000000000000
FETCH	111111	B_DECODE			FETCH
0000000000000000	0000000000000010				0000000000000000
FETCH	010000	BEQ_DECODE	BEQ_EXECUTE		FETCH
0000000000000000	0000010000000010	001(1/0)10000000100			0000000000000000
FETCH	010001	BNE_DECODE	BNE_EXECUTE		FETCH
0000000000000000	0000010000000010	001(0/1)10000000100			0000000000000000
FETCH	000011	LB_DECODE	LB_EXECUTE	LB_WB	FETCH
0000000000000000	0000000000000001	0100000100000000			0000000000000000
FETCH	001111	LW_DECODE	LW_EXECUTE	LW_WB	FETCH
0000000000000000	0000000000000001	0000000100000000			0000000000000000
FETCH	011111	SW_DECODE	SW_EXECUTE		FETCH
0000000000000000	0000000000000001	0000010100000000			0000000000000000

Το διάνυσμα εξόδου V που παρατίθεται παραπάνω, σε κάθε state, μεταφράζεται ως εξής:

V(0): RF_WrEn

V(1): isLoadByte

V(2): isConditionalBranch

V(3): PC_Sel

V(4): PC_LdEn

V(5): RF_B_sel

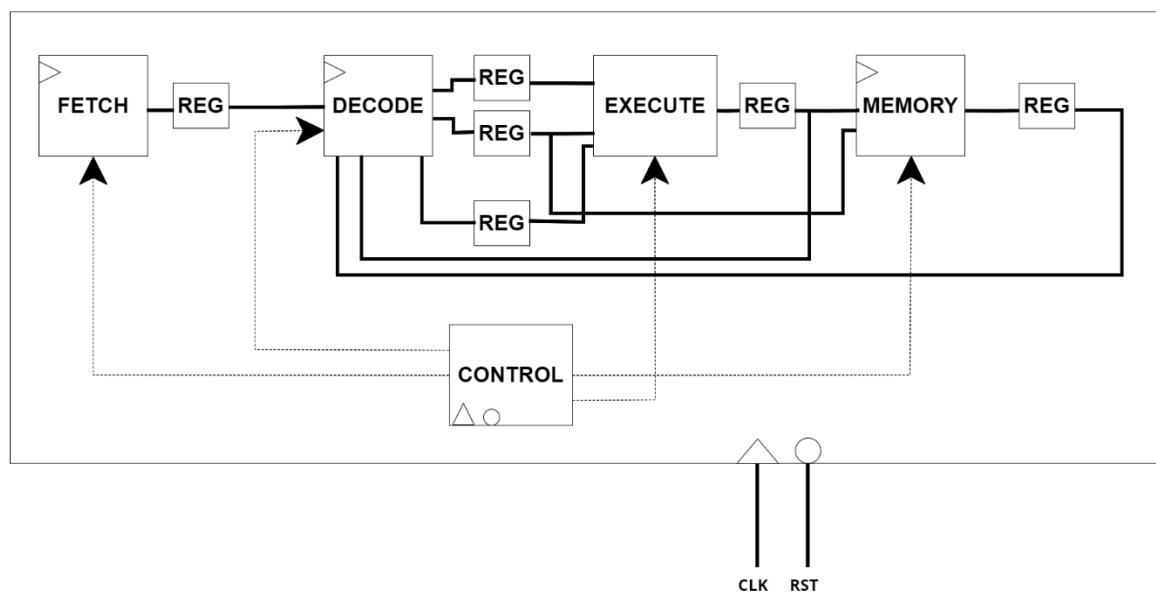
V(6): RF_WrData_sel

V(7): ALU_BIn_sel

V(8): MEM_WrEn

V(12 [downto](#) 9): ALU_func

V(14 [downto](#) 13): immed_operation



Block Diagram του κυκλώματος

Αναφορά αποτελεσμάτων – Επιβεβαίωση λειτουργίας

Αφού φορτωθεί το .coe αρχείο αρχικοποίησης της μνήμης εντολών, θεωρητικά αρχίζει να εκτελείται το παρακάτω MIPS Assembly πρόγραμμα:

```
Showcase Assembly Program
1 li $1,6
2 li $2,6
3 add $1,$3,$2
4 bne $1,$2, 0
5 beq $1,$2, 0
6
7 #####
8
9 lui $8,4
10 lui $10,16
11 or $8,$5,$10
12 rol $5,$6,1
13 ror $6,$5,1
14 sw $5,4($10)
15
16 #####
17
18 sub $5,$9,$1
19 and $5,$11,$8
20 B 4
21 not $1,$4
```

Μετά από λίγη μελέτη των εντολών του προγράμματος, καθώς και των καταχωρητών που συμμετέχουν σε αυτές, το σύστημα θα πρέπει να έχει αποθηκεύσει τις εξής πληροφορίες:

- $\$1 = 6_{10}$
- $\$2 = 6_{10}$
- $\$3 = 12_{10}$
- $\$8 = 262.144_{10} = 0000\ 0000\ 0000\ 0100\ 0000\ 0000\ 0000\ 0000_2$
- $\$10 = 1.048.576_{10} = 0000\ 0000\ 0001\ 0000\ 0000\ 0000\ 0000\ 0000_2$
- $\$5 = 1.310.720_{10} = 0000\ 0000\ 0001\ 0100\ 0000\ 0000\ 0000\ 0000_2$
- $\$6 = 2.621.440_{10} = 0000\ 0000\ 0010\ 1000\ 0000\ 0000\ 0000\ 0000_2$
- $\$9 = 262.138_{10} = 0000\ 0000\ 0000\ 0011\ 1111\ 1111\ 1111\ 1010_2$
- $\$11 = 262.144_{10} = 0000\ 0000\ 0000\ 0100\ 0000\ 0000\ 0000\ 0000_2$
- $\$4 = 0$, καθώς λόγω της B, η not της γραμμής 21 παραλείπεται.
- $\text{MEM}[1] = 1.310.720_{10} = 0000\ 0000\ 0001\ 0100\ 0000\ 0000\ 0000\ 0000_2$

Πράγματι, ελέγχοντας τις τελικές τιμές των καταχωρητών και της μνήμης δεδομένων, μετάτο πέρας του Simulation, οι προαναφερθείσες τιμές επιβεβαιώνονται.

Συμπεράσματα

Ολοκληρώνοντας την άσκηση, ο φοιτητής έκανε τη μετάβαση από το μοντέλο του επεξεργαστή ενός κύκλου, στο πιο ρεαλιστικό μοντέλο πολλαπλών κύκλων που αξιοποιείται σήμερα, και διδάσκεται και στα πλαίσια του μαθήματος. Τέλος, του δόθηκε η δυνατότητα να χρησιμοποιήσει τις γνώσεις που διδάχθηκε στη θεωρία αυτού του μαθήματος, αλλά και απαραίτητων προαπαιτούμενων του, εφαρμόζοντας παράλληλα, καλές πρακτικές σχεδίασης υλικού.

Συνολικός χρόνος ενασχόλησης με την άσκηση:

Γράψιμο κώδικα, τρέξιμο Simulation, debugging, διόρθωση πρώτης φάσης 35 ώ
+ Σύνταξη αναφοράς 3 ώ = **38 ώ**